

IF332

FORMAL LANGUAGES

&

AUTOMATA THEORY

02

DETERMINISTIC FINITE AUTOMATA (DFA)

DENNIS GUNAWAN

REVIEW

Introduction:

Automata Theory

Symbols, Alphabets, Strings, Languages

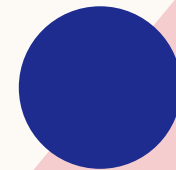
OUTLINE

Finite State Automata (FSA)

Deterministic Finite Automata (DFA)

Regular Languages

Equivalence of DFA



FINITE STATE AUTOMATA

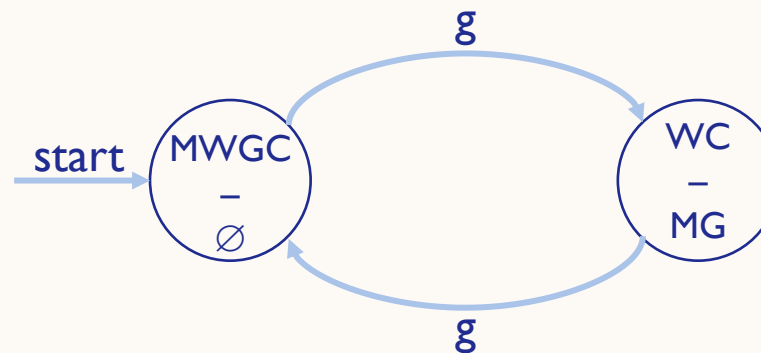
- It is the simplest model of computation
- It has a very limited memory
- Example
 - Elevator / lift control mechanism
 - Text editor
 - The classic river crossing puzzle: the wolf, the goat, and the cabbage



THE CLASSIC RIVER CROSSING PUZZLE

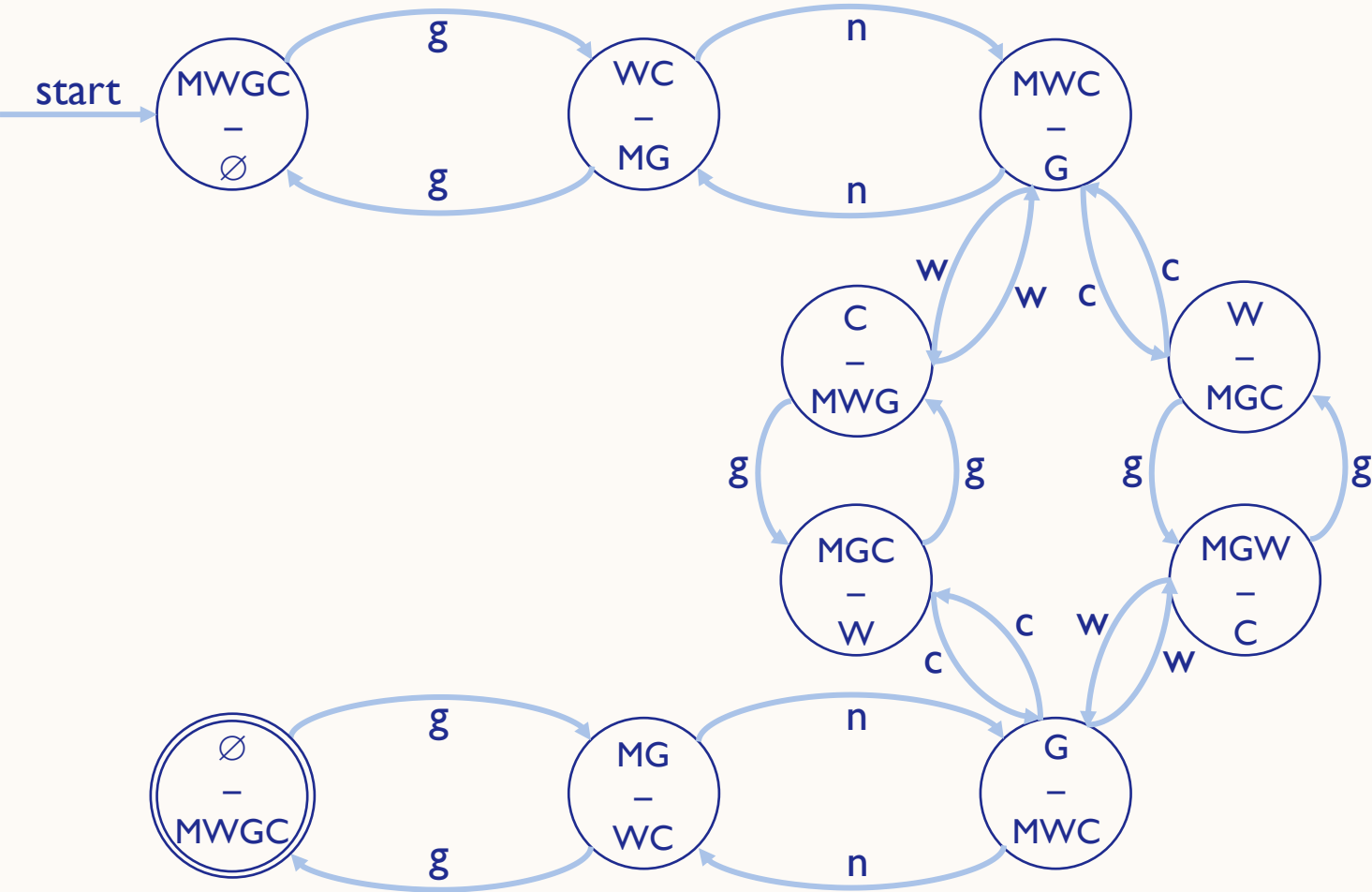
THE WOLF, THE GOAT, AND THE CABBAGE

- Moves as symbols, solutions as strings
 - w: man crosses with wolf
 - g: man crosses with goat
 - c: man crosses with cabbage
 - n: man crosses with nothing
- Moves as state transitions
 - A state is the configuration of occupants on each side of the river



THE CLASSIC RIVER CROSSING PUZZLE

THE WOLF, THE GOAT, AND THE CABBAGE

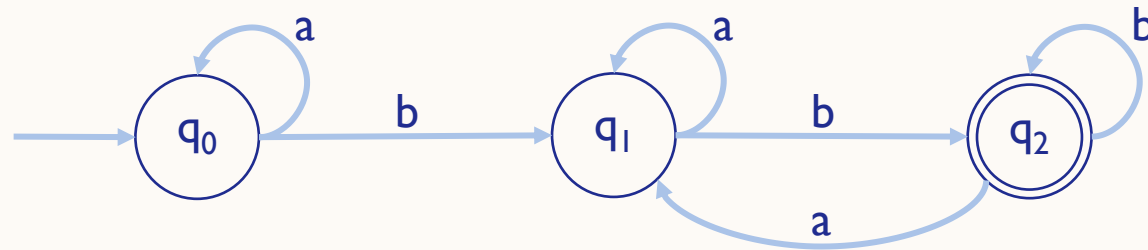


DETERMINISTIC FINITE AUTOMATA (DFA)

- The term “deterministic” refers to the fact that on each input there is **one and only one state** to which the automaton can transition from its current state
- Five-tuple notation: $M = (Q, \Sigma, \delta, q_0, F)$
 - Q : a set of states
 - Σ : a set of input symbols
 - δ : a transition function from $Q \times \Sigma \rightarrow Q$
 - q_0 : a start state
 - F : a set of final or accepting states

SIMPLER NOTATIONS FOR DFA

Transition Diagram



Transition Table

δ	a	b
q ₀	q ₀	q ₁
q ₁	q ₁	q ₂
q ₂	q ₁	q ₂

$\delta(q_0, a) = q_0$
 $\delta(q_0, b) = q_1$
 $\delta(q_1, a) = q_1$
 $\delta(q_1, b) = q_2$
 $\delta(q_2, a) = q_1$
 $\delta(q_2, b) = q_2$

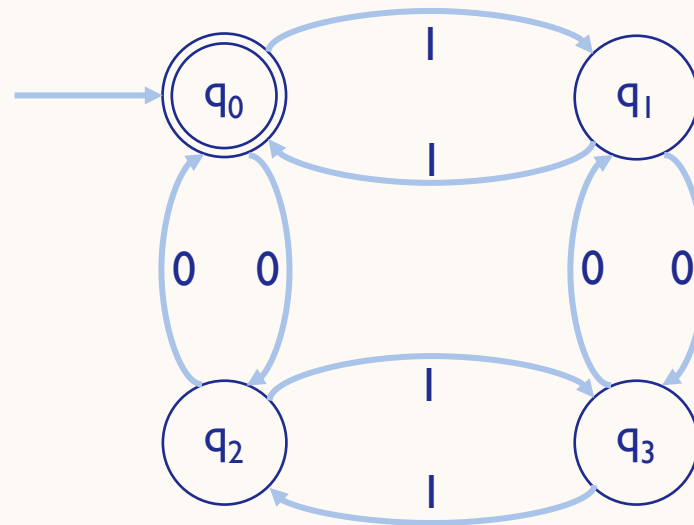
$Q = \{q_0, q_1, q_2\}$
 $\Sigma = \{a, b\}$
 $q_0 = q_0$
 $F = \{q_2\}$

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$

EXAMPLE

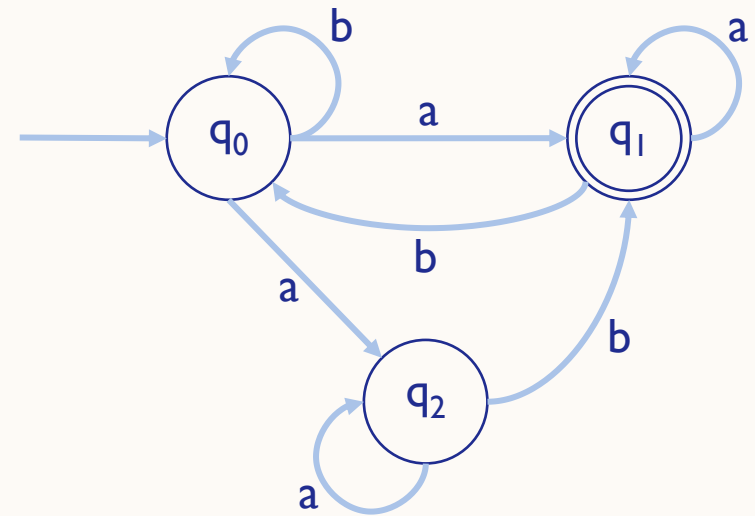
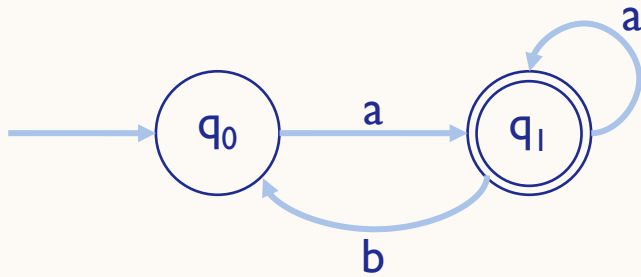
Design a DFA to accept the language

$L = \{w \mid w \text{ has both an even number of 0's and an even number of 1's}\}$



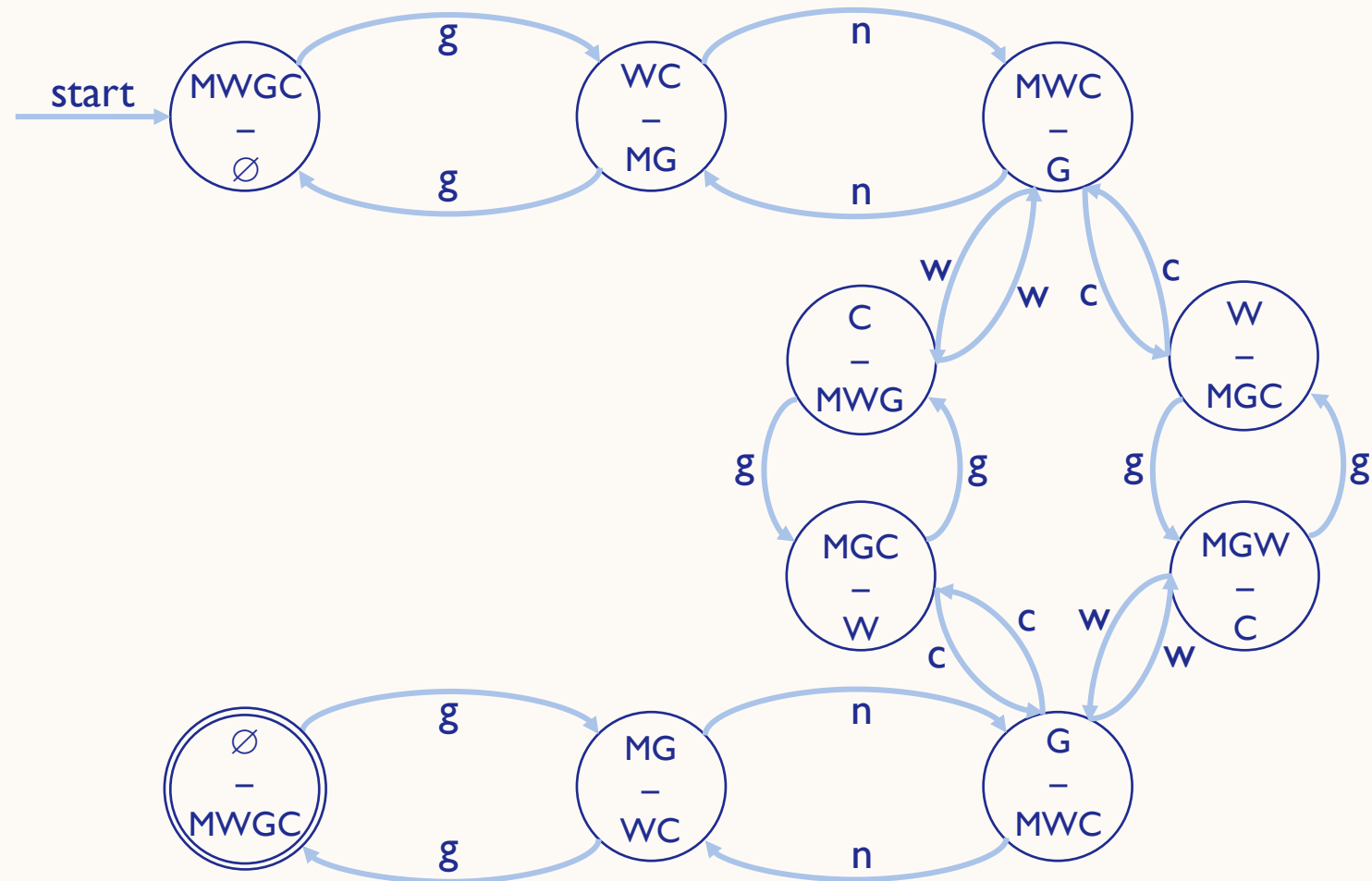
$$M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\})$$

NON-DFA



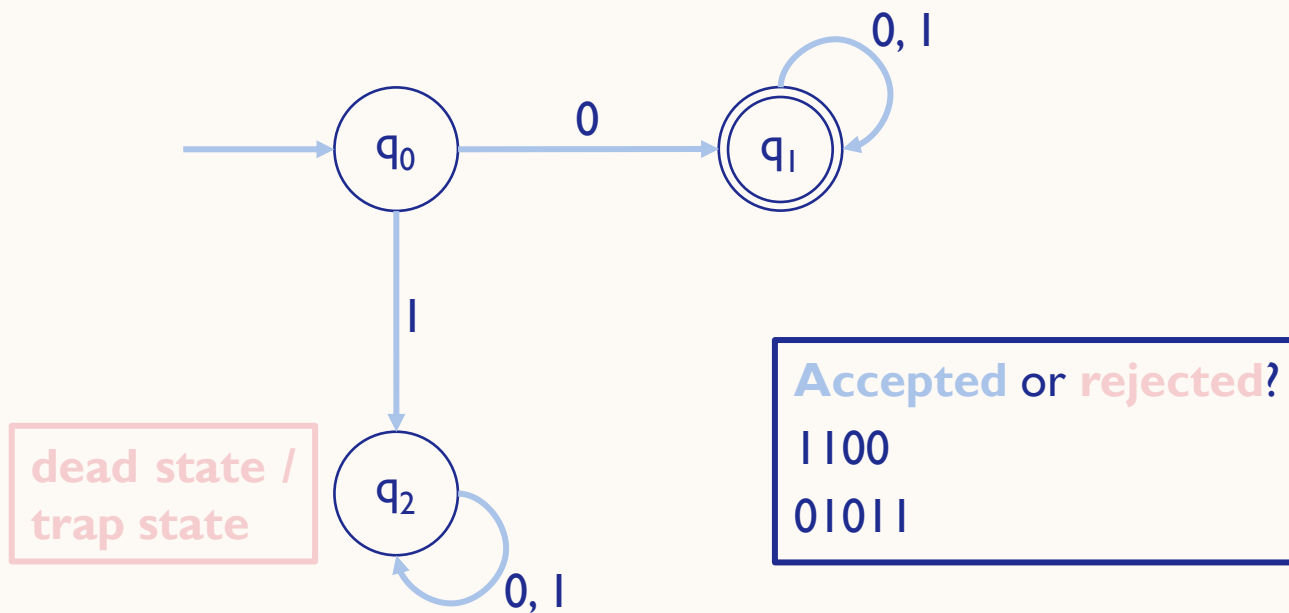
WHY?

DFA?



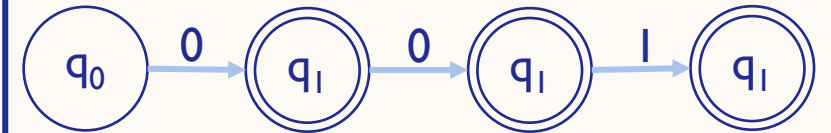
EXAMPLE

Construct a DFA that accepts any strings over $\{0, 1\}$ that start with '0'



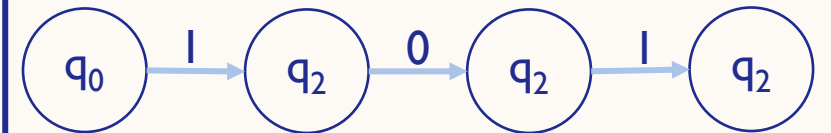
001 : **accepted**

$$\delta(q_0, 001) = \delta(q_1, 01) = \delta(q_1, 1) = q_1$$



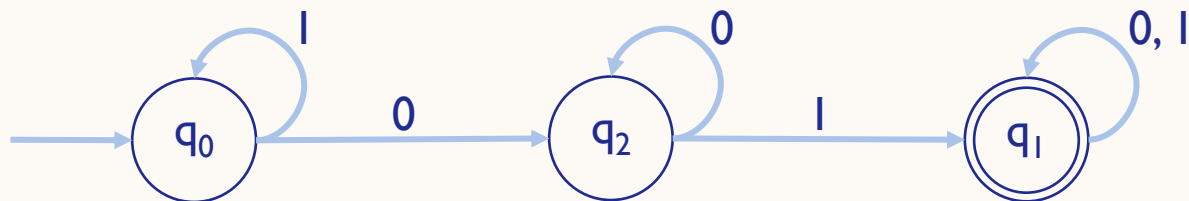
101 : **rejected**

$$\delta(q_0, 101) = \delta(q_2, 01) = \delta(q_2, 1) = q_2$$



THE LANGUAGE OF A DFA

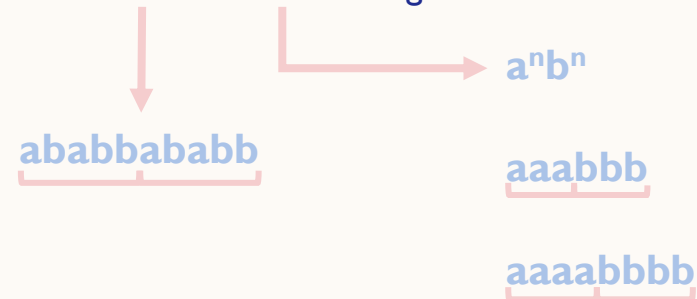
- The language of a DFA $M = (Q, \Sigma, \delta, q_0, F)$ is denoted $L(M)$
- $L(M) = \{w \mid \delta(q_0, w) \text{ is in } F\}$
 - The language of M is the set of strings w that take the start state q_0 to one of the accepting states
- If L is $L(M)$ for some DFA M , then L is a regular language
- Example:



$L(M)$ is the set of all strings of 0's and 1's that contain a substring 01

REGULAR LANGUAGES

- A language is said to be a REGULAR LANGUAGE if and only if some Finite State Machine recognizes it
- What languages are NOT REGULAR?
 - The languages which are not recognized by any FSM
 - The languages which require memory
 - Memory of FSM is very limited
 - It cannot store or count strings

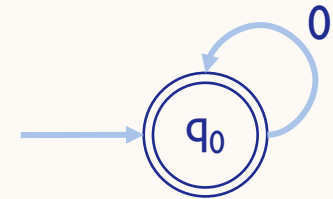
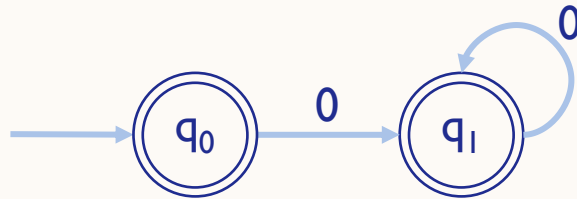


OPERATIONS ON REGULAR LANGUAGES

- UNION $: A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
- CONCATENATION $: A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
- STAR $: A^* = \{x_1 x_2 x_3 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$
- $A = \{pq, r\}$ $B = \{t, uv\}$
 - $A \cup B = \{pq, r, t, uv\}$
 - $A \circ B = \{pqt, pquv, rt, ruv\}$
 - $A^* = \{\epsilon, pq, r, pqpq, pqr, rpq, rr, pqpqpq, rrr, \dots\}$

EQUIVALENCE OF DFA

- Two DFA's are said to be equivalent if they accept the same language
 - $L(M_1) = L(M_2)$



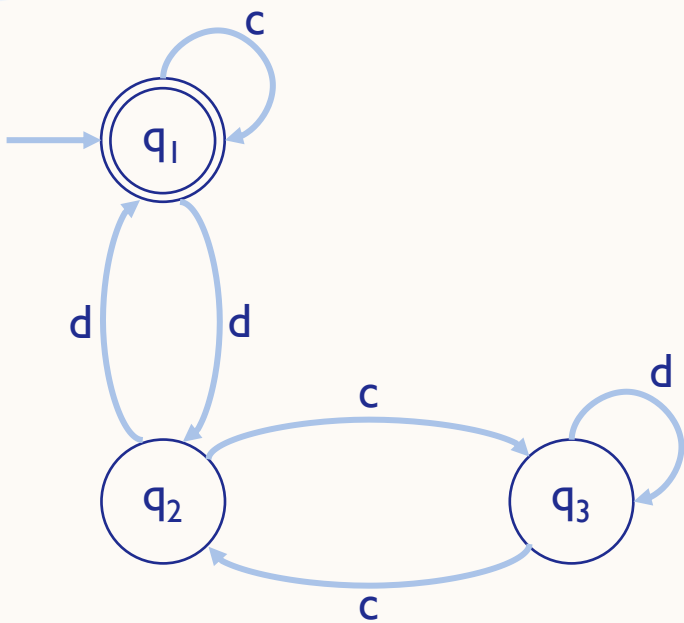
EQUIVALENCE OF DFA

- Steps to identify equivalence
 - For any pair of states (q_i, q_j) , the transition for input $a \in \Sigma$ is defined by (q_a, q_b) where $\delta(q_i, a) = q_a$ and $\delta(q_j, a) = q_b$

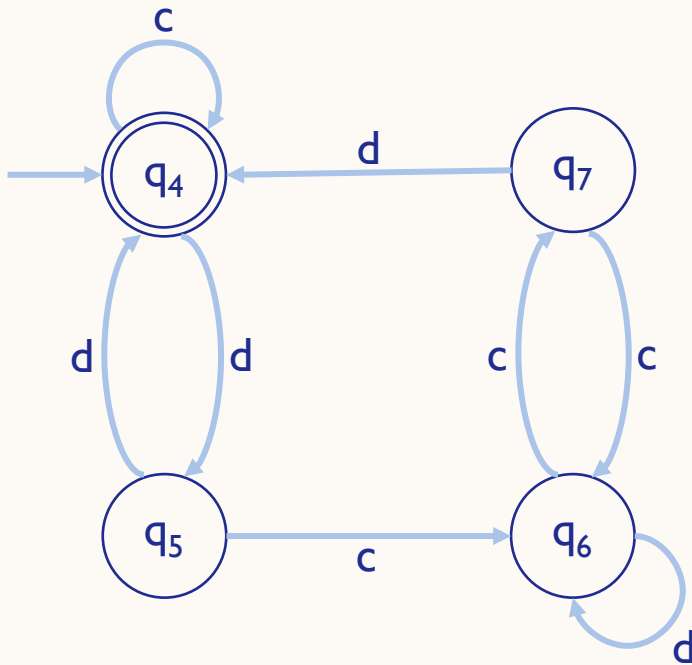
The two automata are NOT equivalent if for a pair (q_a, q_b) , one is INTERMEDIATE (nonaccepting) state and the other is FINAL (accepting) state

- If the initial state is also the final state of one automaton, then in the second automaton, the initial state must also be the final state for them to be equivalent

EXAMPLE



A



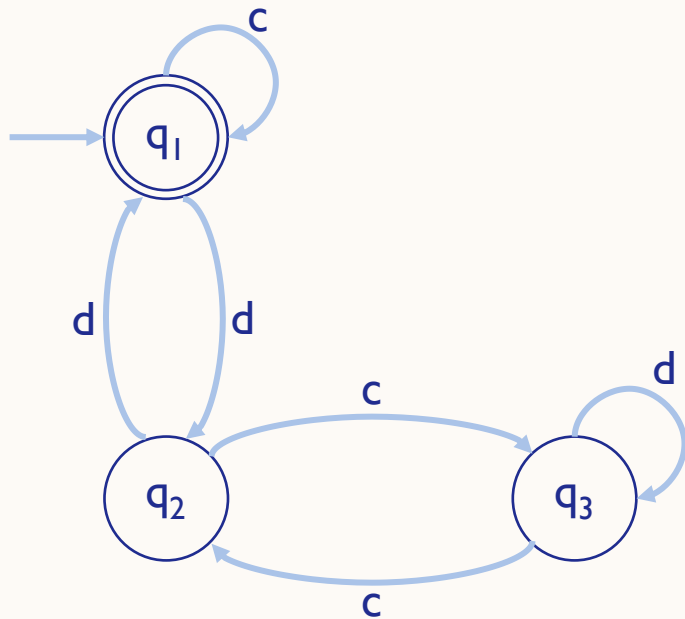
B

A and B are equivalent

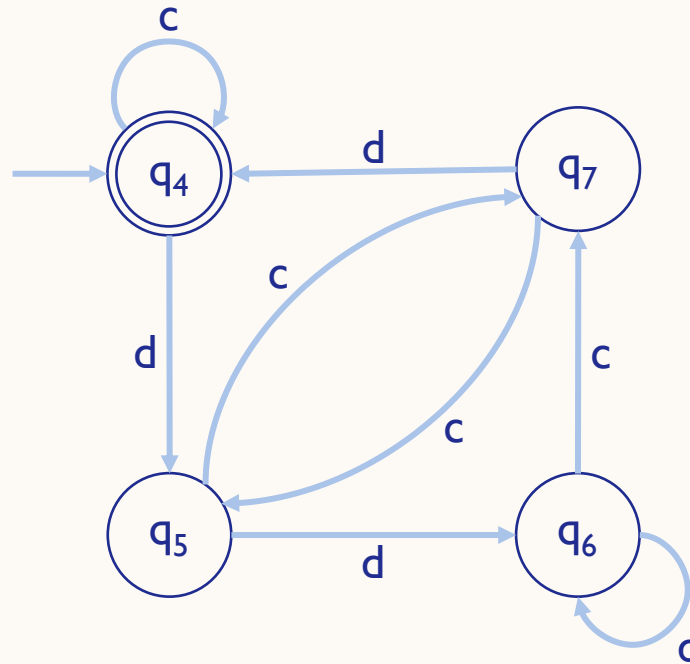
States	c	d
(q ₁ ,q ₄)	(q ₁ ,q ₄) F F	(q ₂ ,q ₅) I I
(q ₂ ,q ₅)	(q ₃ ,q ₆) I I	(q ₁ ,q ₄) F F
(q ₃ ,q ₆)	(q ₂ ,q ₇) I I	(q ₃ ,q ₆) I I
(q ₂ ,q ₇)	(q ₃ ,q ₆) I I	(q ₁ ,q ₄) F F

F : Final (accepting) state
I : Intermediate (nonaccepting) state

EXAMPLE



A



B

A and B are not equivalent

States	c	d
(q_1, q_4)	(q_1, q_4) F F	(q_2, q_5) I I
(q_2, q_5)	(q_3, q_7) I I	(q_1, q_6) F I

F : Final (accepting) state

I : Intermediate (nonaccepting) state



PRACTICE

EXERCISES

- I. Give the transition diagram for the DFA below.

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

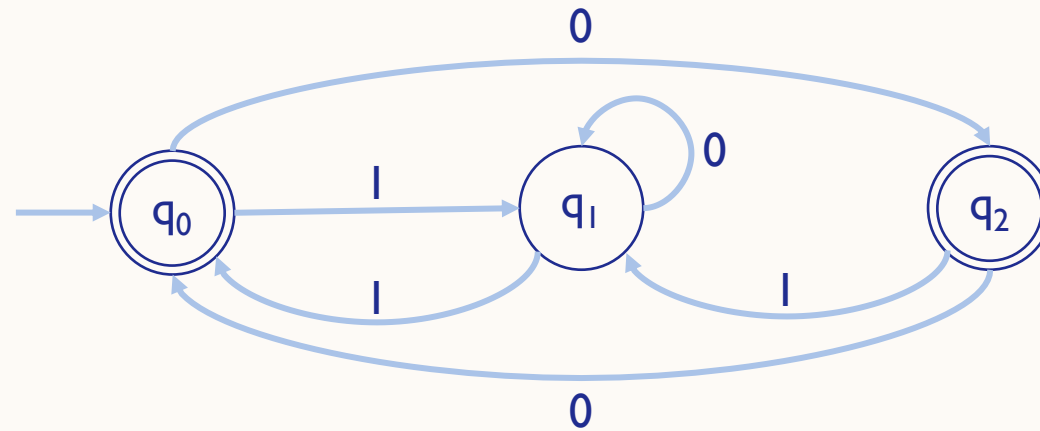
$$q_0 = q_0$$

$$F = \{q_0\}$$

δ	a	b
q_0	q_1	q_2
q_1	q_2	q_0
q_2	q_2	q_2

EXERCISES

2. Give the transition table for the DFA below.



Accepted or rejected?

a. 1010

b. 1101

c. 0010

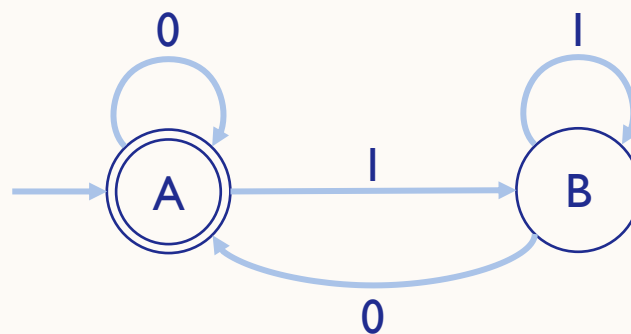
d. 0101

EXERCISES

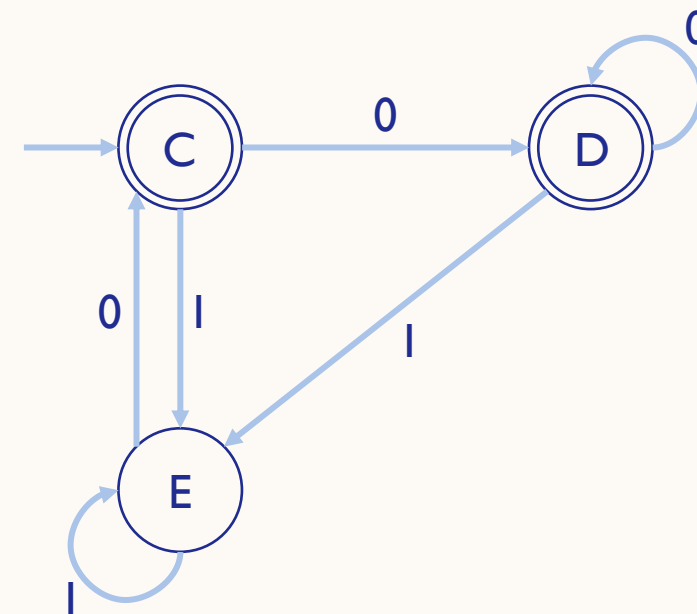
3. Design a DFA accepting set of all strings over $\{a, b\}$ of length 2.
4. Construct a DFA that accepts set of all strings over $\{0, 1\}$ ending in '00'.
5. Construct a DFA that accepts any strings over $\{0, 1\}$ that contain the string '0011' in it.
6. Construct a DFA that accepts any strings over $\{0, 1\}$ that do not contain the string '0011' in it.
7. Design a DFA accepting set of all strings over $\{0, 1\}$ that, when interpreted as a binary integer, is a multiple of 5.

EXERCISES

8. Are A and B equivalent?



A



B

REFERENCES

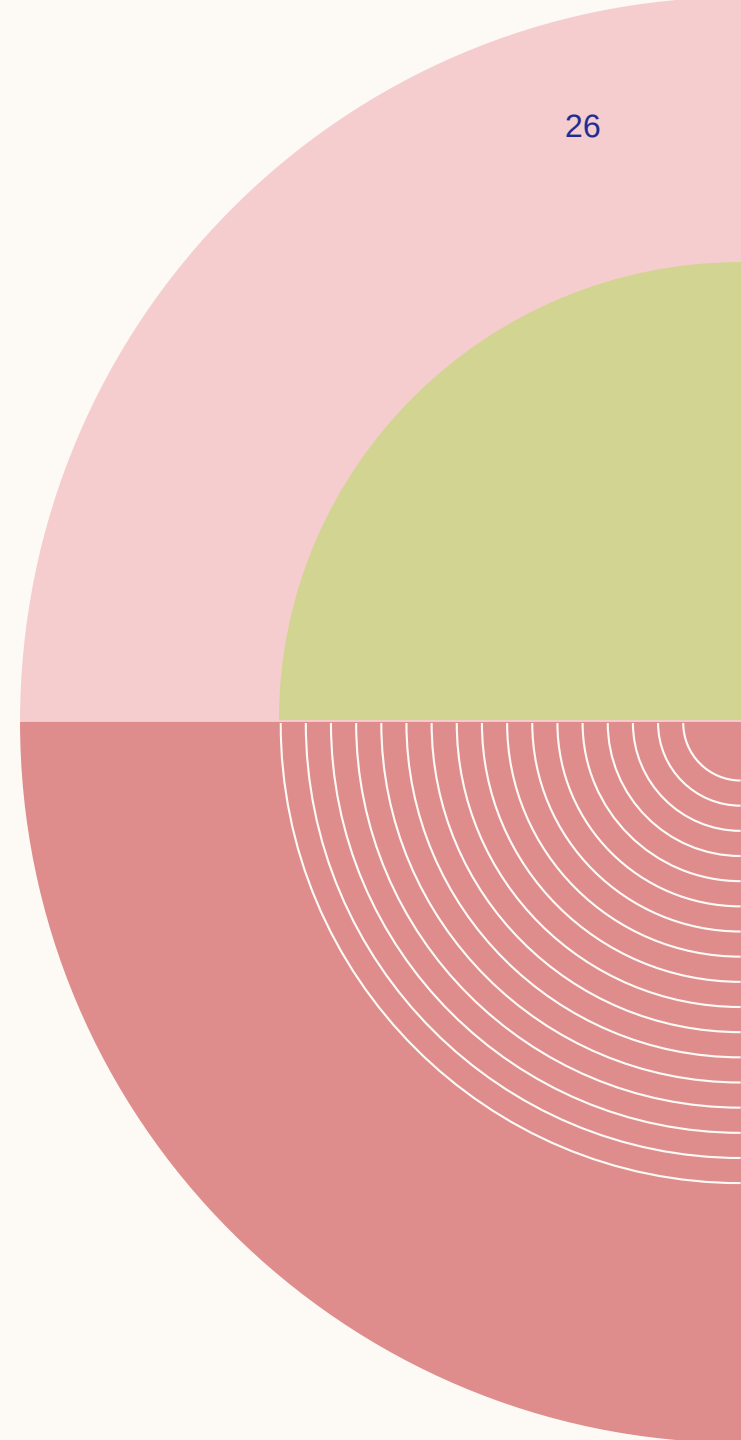
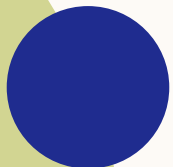
- Utdirartatmo, F. (2005), Teori Bahasa dan Otomata (edisi ke-2), Graha Ilmu.
- Hopcroft, John E., Rajeev Motwani, and Jeffrey D. Ullman (2006), Introduction to Automata Theory, Languages, and Computation (3rd edition), Addison-Wesley.

NEXT

Nondeterministic Finite Automata (NFA):

Nondeterministic Finite Automata (NFA)

Conversion of NFA to DFA



VISION

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.

MISSION

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology



iF INFORMATIKA
UMN